UEPB
Universidade Estadual da Paraíba
Centro de Ciências e Tecnologia – CCT
Departamento de Computação - DC

DISCIPLINA: LABORATÓRIO DE OAC – LOAC

RELATÓRIO 01: Modos de Endereçamento e Funções do MCU51

(2021)

ALUNO: Dirceu Araújo Macêdo

Prof(ª). Dr(ª). Misael Elias de Morais

CAMPUS CAMPINA GRANDE, Julho de 2021.

RESUMO

Este experimento visa a realização do uso de pseudo-instruções e instruções do Assembly, para verificar o comportamento dessas instruções no simulador. Onde é apresentado algumas as situações que devem ser executadas no microcontrolador MCU8051 e seus comandos (conjunto de instruções). As atividades sugeridas pelo professor foram realizadas e apresentadas neste relatório para permitir que a documentação e validação da prática e o conhecimento do conjunto de instruções aplicadas no microcontrolador.

Palavras-chave: Microcontrolador. Assembly. MCU8051

MATERIAL E MÉTODOS

Para aprender e praticar o experimento, o professor Misael Morais fez vídeo chamadas síncronas e cursos assíncronos de 01 a 12. Foi utilizado o material fornecido por ele em seu site (MORAIS, 2020), que contém instruções passo a passo sobre a instalação do simulador MCU 8051 IDE e as atividades realizadas neste relatório. Finalmente, usamos a emulação do microcontrolador MCU51, chamada MCU8051 UD, e praticamos o conjunto de instruções fornecido pelo professor para obter este relatório.

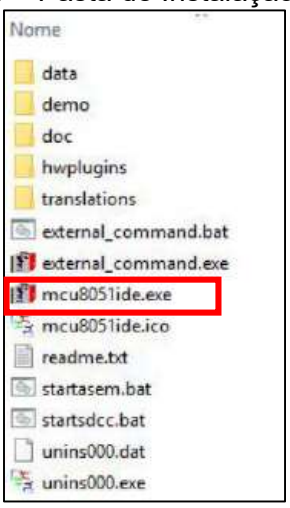
RESULTADOS E DISCUSSÂO

Instalação e Execução do Programa

Instalação

A instalação da IDE pode ser feita manualmente ou executando o SETUP.EXE disponível no site da disciplina. Esta instalação criará a seguinte estrutura de pasta.

Figura 1 – Pasta de Instalação Modelo

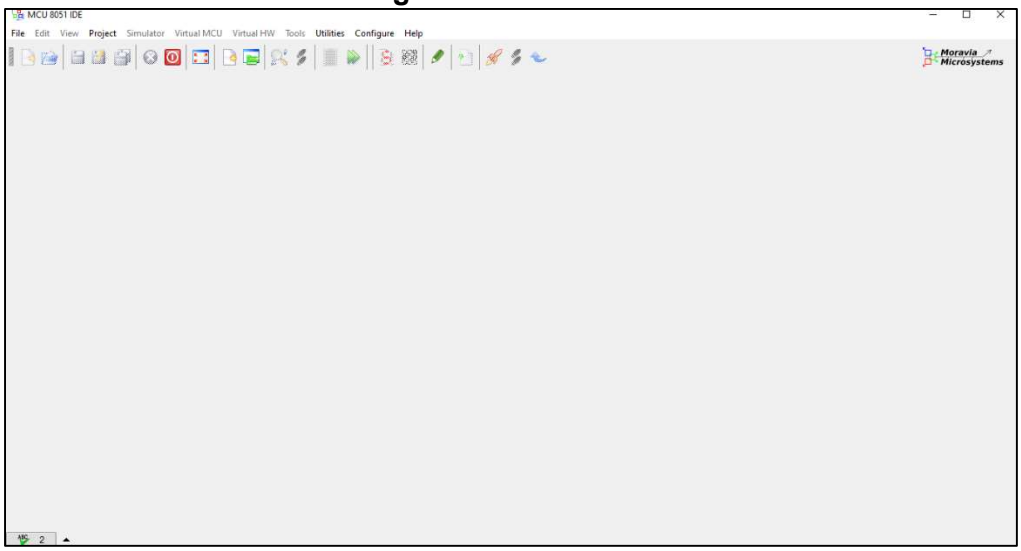


O arquivo mcu8051ide.exe é o executável desta IDE. Não é necessário realizar nenhuma instalação, somente abrir a pasta descompactada e executar “mcu8051ide.exe”.

Execução do Programa

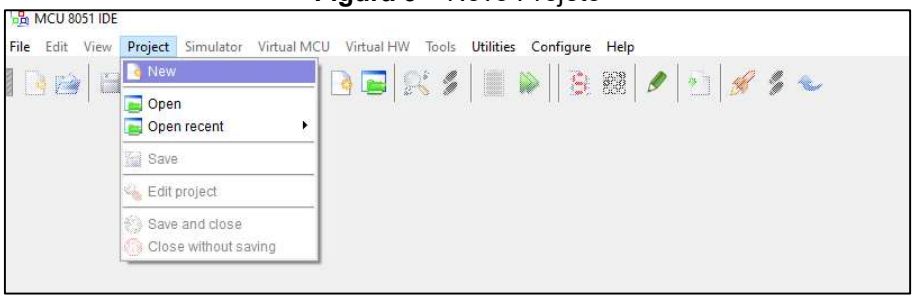
Na execução do programa o usuário irá se deparar com a situação ilustrada na figura 2. Onde deverá criar um novo projeto para acessar os recursos do programa.

Figura 2 – Tela Inicial



Para iniciar um novo projeto o usuário deve clicar em *Project* e em seguida em *New*, conforme ilustrado na Figura 3 abaixo.

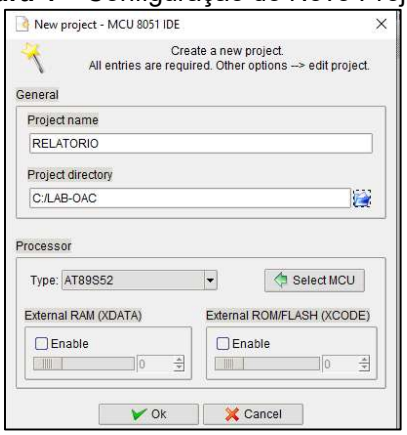
Figura 3 – Novo Projeto



Ao clicar no botão *New* o utilizador irá se deparar com a tela ilustrada na figura 4. Onde deverá preencher os seguintes campos *Project Name* e o *Project directory*. O *Project Name* se refere ao campo do nome do projeto e o *Project directory* o local onde o projeto será salvo.

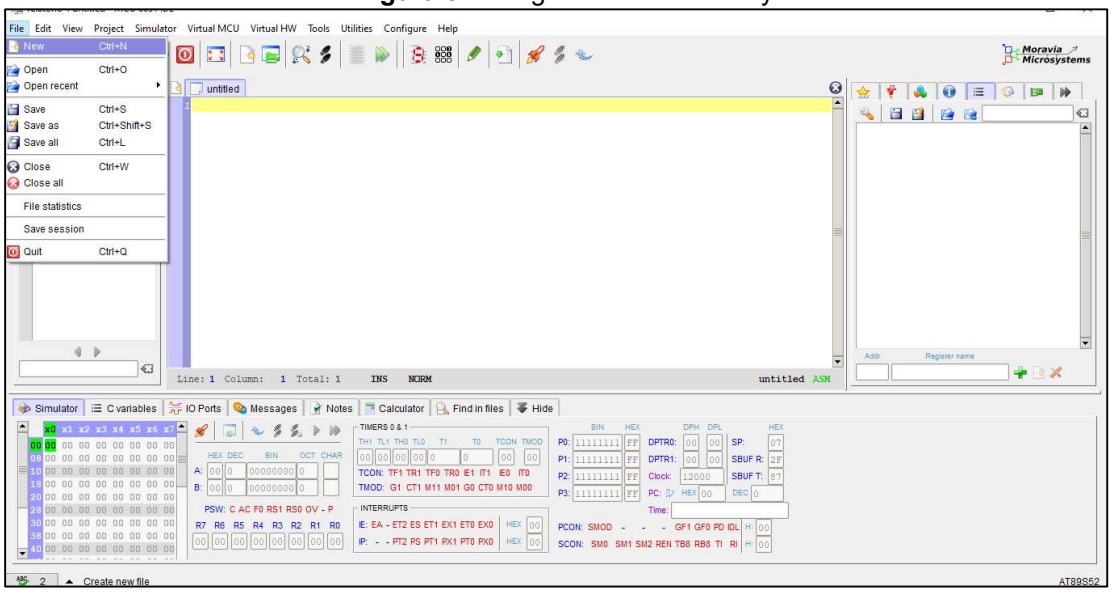
Na parte do *Processor* o *Type* MCU utilizado será o “AT89S52”.

Figura 4 – Configuração do Novo Projeto.



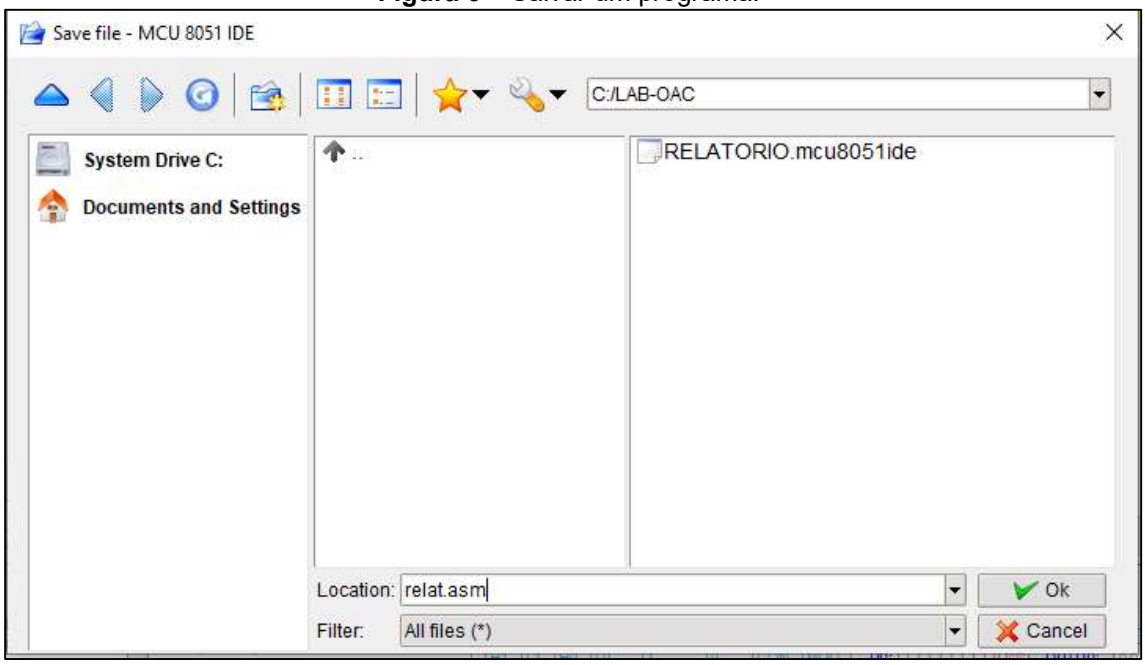
Para criar um novo programa em Assembly, basta ir em *File* e em seguida em *New, e* irá aparecer um editor de texto com o nome “untitled” para preencher as instruções dos códigos.

Figura 5 – Programa em Assembly.



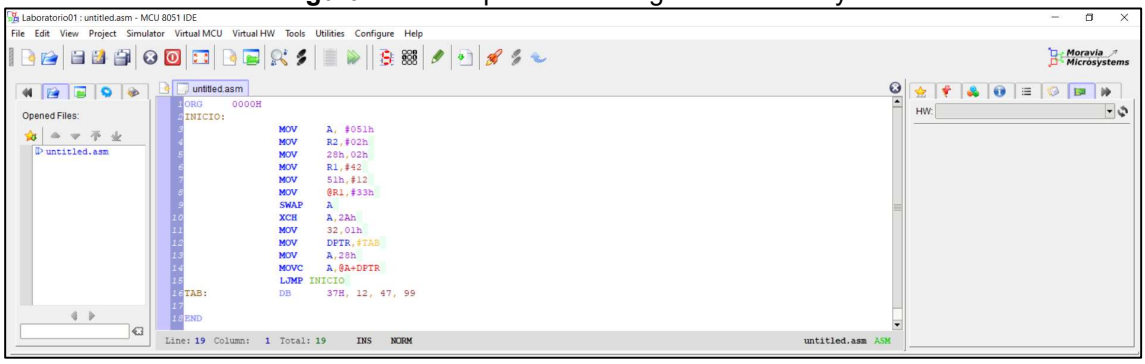
Para salvar o programa basta ir em *File* e em seguida *Save* que o programa salva até a última modificação feita, conforme na figura 6. Após clicar em *Save* será apresentado ao usuário uma nova tela. O arquivo deve ser salvo com o final “.asm” em um diretório próximo a raiz. O nome utilizado deve ser curto.

Figura 6 – Salvar um programa.



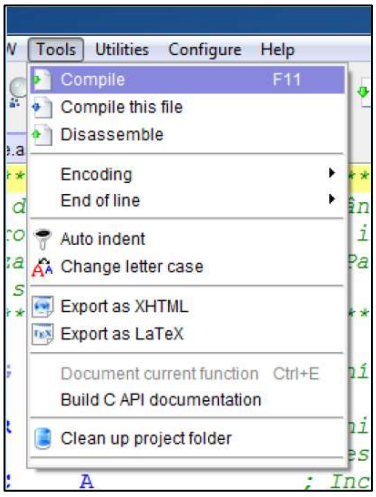
As figuras 7, 8, 9 se referem as condições e tapas do programa assembly ao executar e compilar um determinado conjunto de instruções.

Figura 7 – Exemplo de um Programa Assembly.



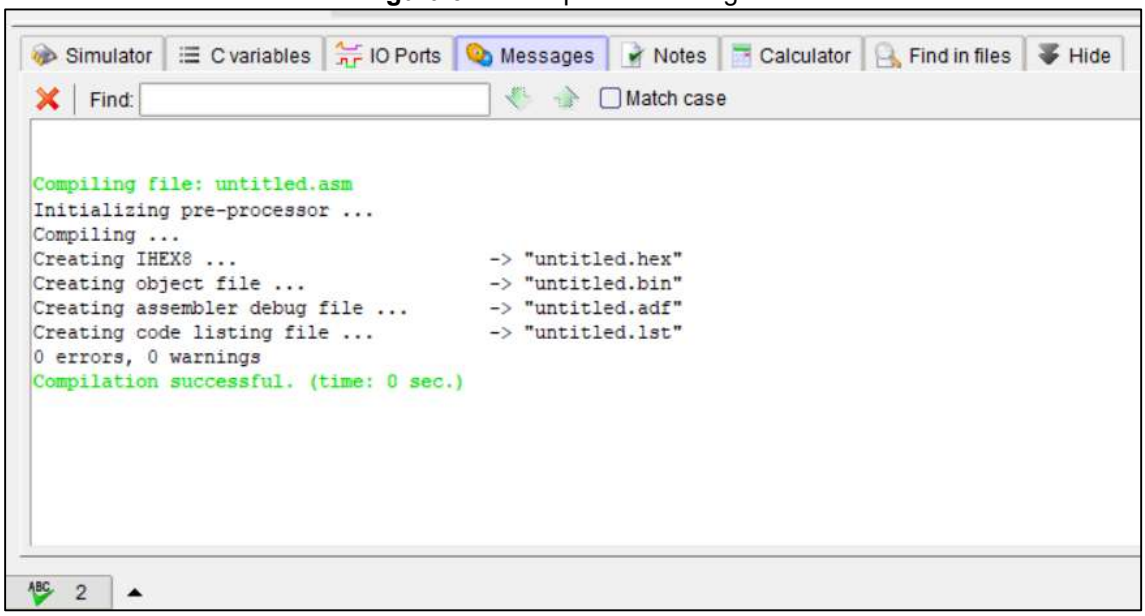
Para compilar o programa assembly é necessário que o usuário siga vá para *Tools* e depois *Compile* (ou pressionar a tecla F11) para começar a compilar.

Figura 8 – Inicio da Compilação.



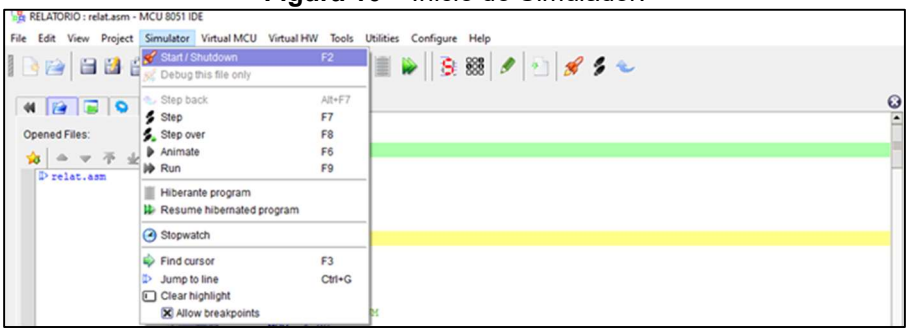
O código objeto é gerado no formato ".hex" no mesmo diretório do arquivo onde o código foi salvo. Para observar se a compilação ocorreu de forma correta o usuário deve ir para mensagens e verificar se houve erros. A figura 9 ilustra, um exemplo de um programa sem erros.

Figura 9 – Exemplo de Mensagem



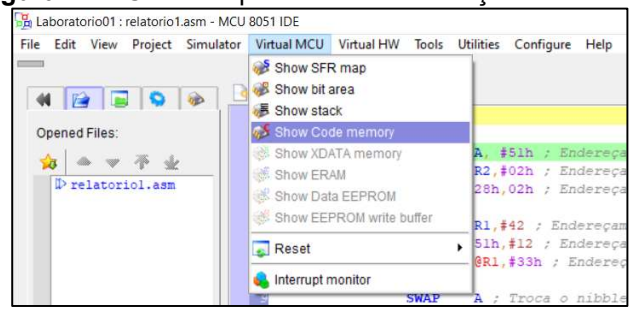
Para visualizar o comportamento das instruções no MCU é necessário iniciar o *Simulator.* Para inicia-lo pressione F2 ou vá em *Simulator* e depois clique em *Start/Shutdown*.

Figura 10 – Inicio do Simulador.



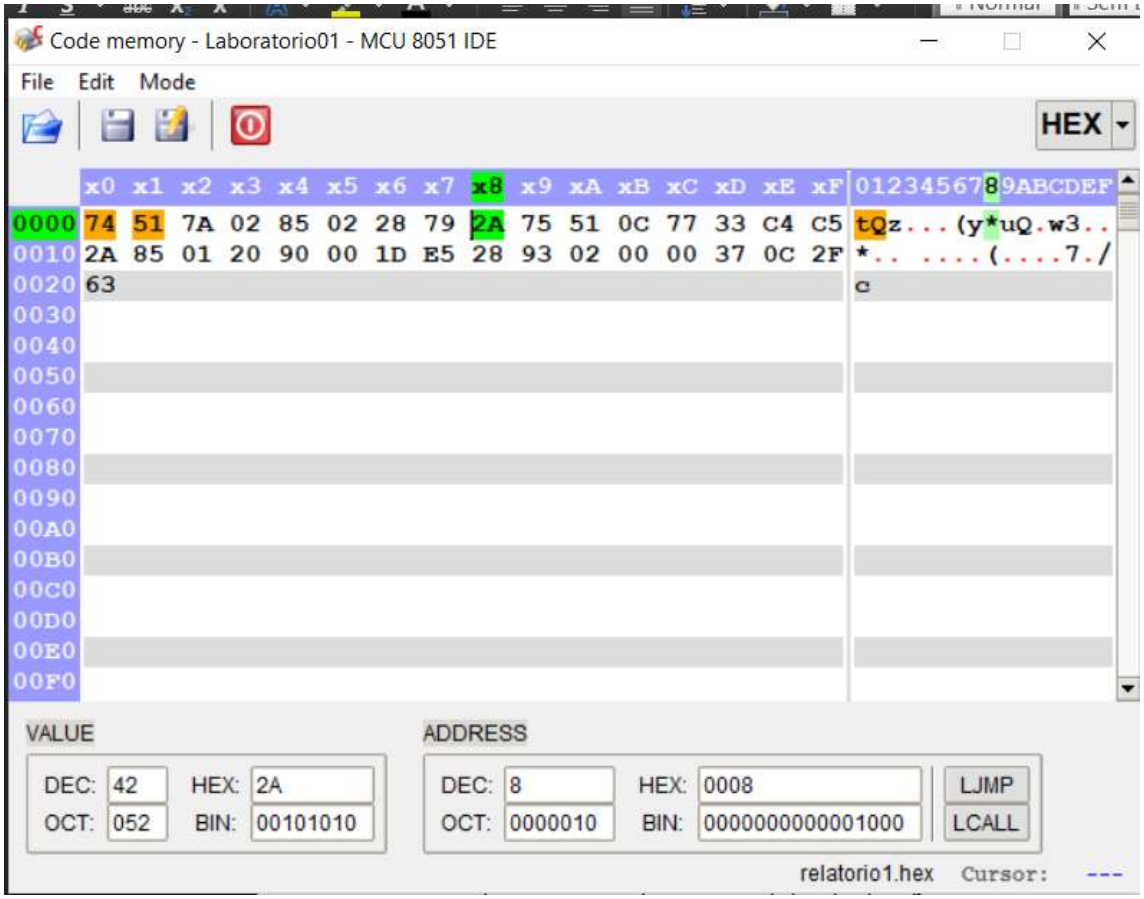
Para acessar identificar o endereço de memória, é só seguir ir em *Virtual MCU* e depois ir em *Show code memory.* Conforme a figura 11.

Figura 11 – Caminho para exibir o endereço de memória.



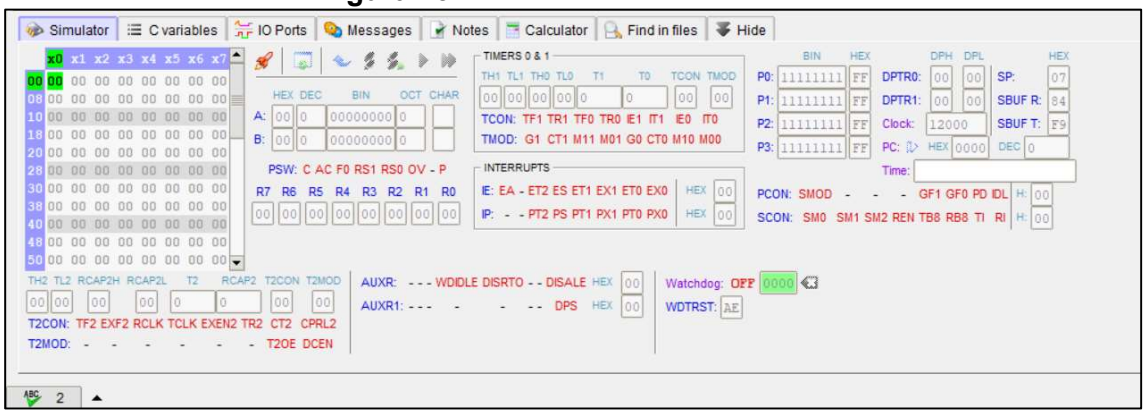
Vai aparecer a seguinte aba, *Code Memory* referente aos registros gravados na memória do programa, conforme a figura 12.

Figura 12 – Memória do Programa



O simulador pode ser utilizado como teste de mesa, onde exibe os valores atuais dos registradores, acumuladores, portas, memórias e etc, conforme a Figura 13.

Figura 13 – Teste de Mesa Simulador



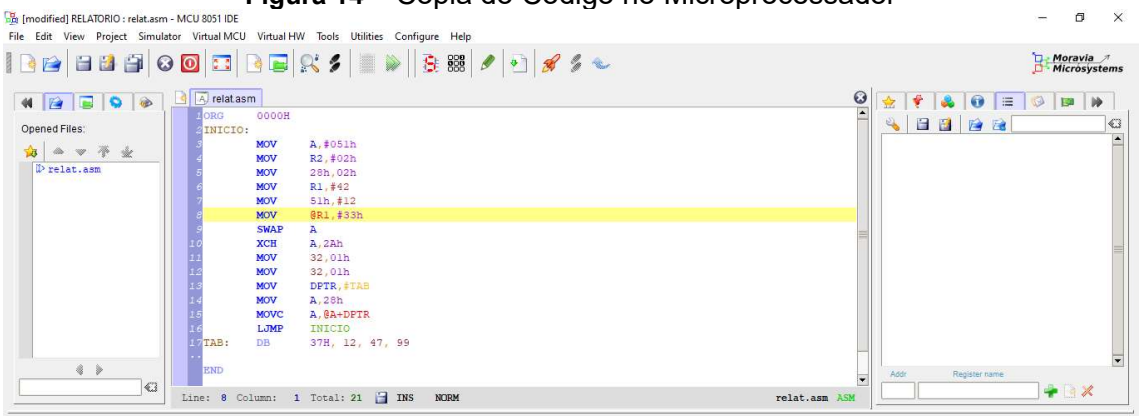
Execução de Instruções Assembly

Com objetivo de analisar o comportamento estabelecido pelo professor, as etapas abaixo se referem a um demonstrativo do resultado obtido ao executar as instruções

1. Edite o programa Assembly abaixo:

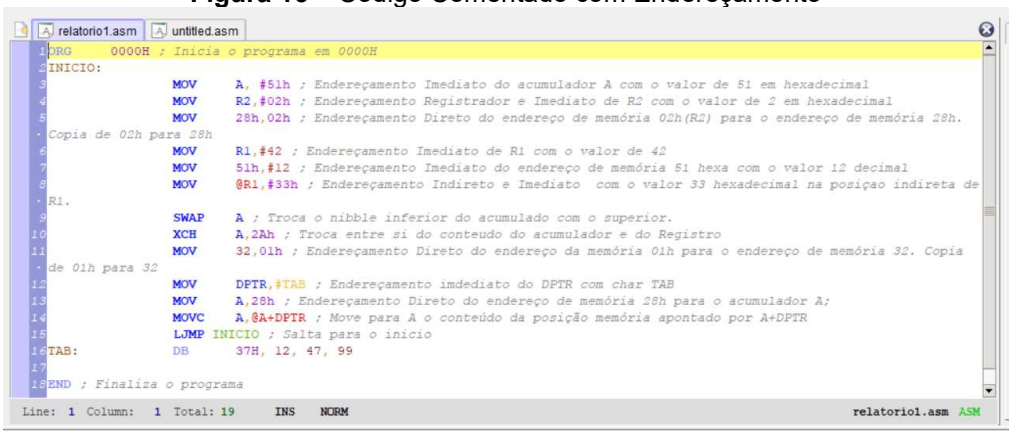
```
ORG     0000H
INICIO:
                MOV     A, #051h
                MOV     R2,#02h
                MOV     28h,02h
                MOV     R1,#42
                MOV     51h,#12
                MOV     @R1,#33h
                SWAP    A
                XCH     A,2Ah
                MOV     32,01h
                MOV     DPTR,#TAB
                MOV     A,28h
                MOVC    A,@A+DPTR
                LJMP INICIO
TAB:            DB      37H, 12, 47, 99

END
```

Figura 14 – Copia do Código no Microprocessador



2. Comente cada linha destacando os modos de endereçamento. Lembrando que o comentário deve iniciar com “;”.

Figura 15 – Código Comentado com Endereçamento



3. Indique o que é mnemônica e operando:

Mnemônica:

MOV A,#051h

MOV R2,#02h

MOV 28h,02h

MOV R1,#42

MOV 51h,#12

MOV @R1,#33h

MOV 32,01h

MOV 32,01h

MOV DPTR,#TAB

MOV A,28h

MOVC A,@A+DPTR

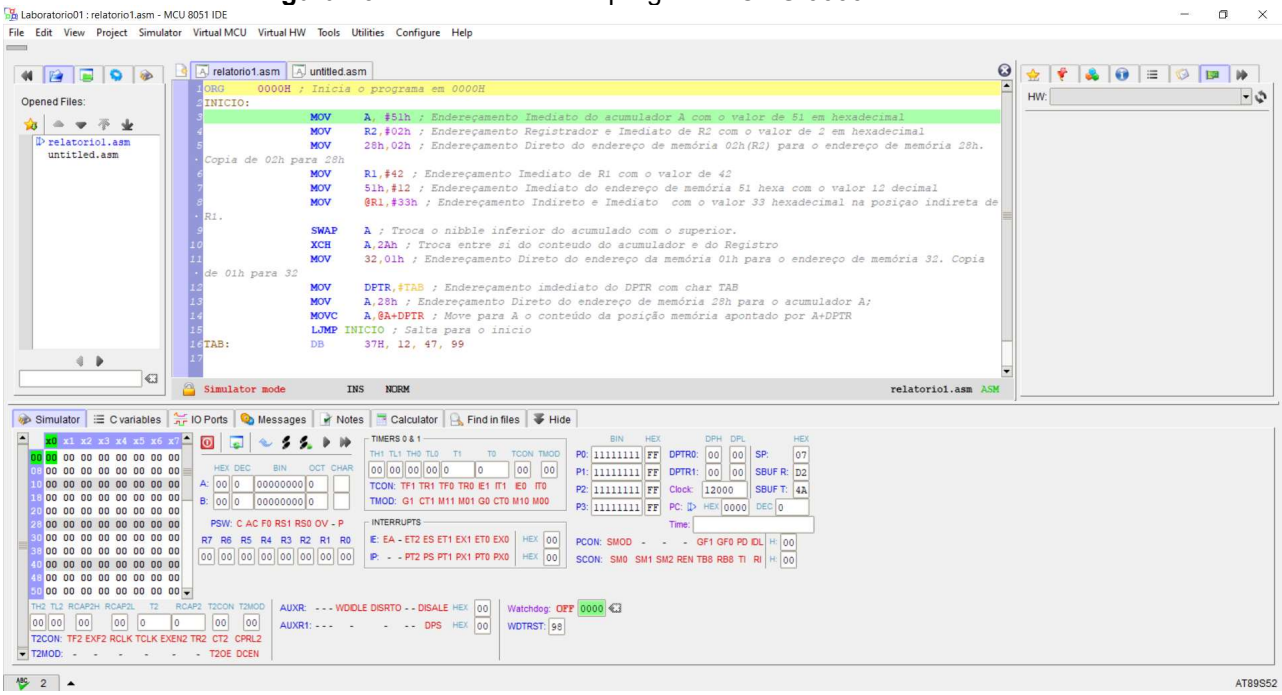
Operando:

LJMP INICIO

TAB: DB 37H, 12, 47, 9

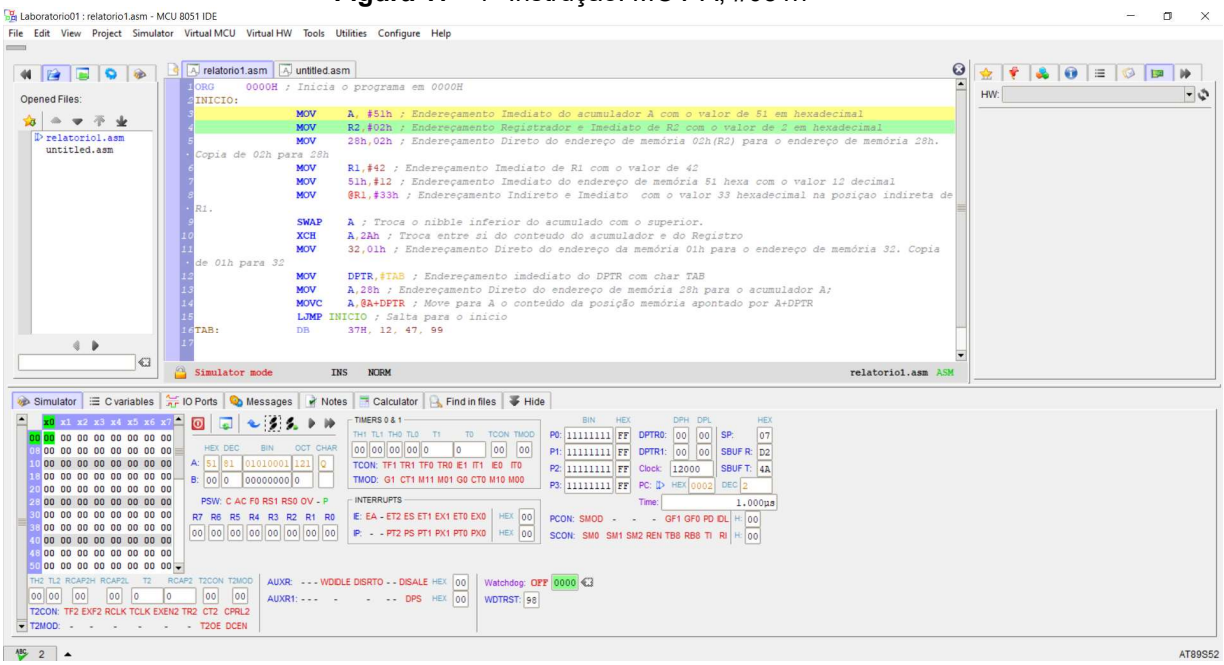
5. Execução e Descrição de cada linha

Figura 16 – Estado Inicial do programa: ORG 0000H



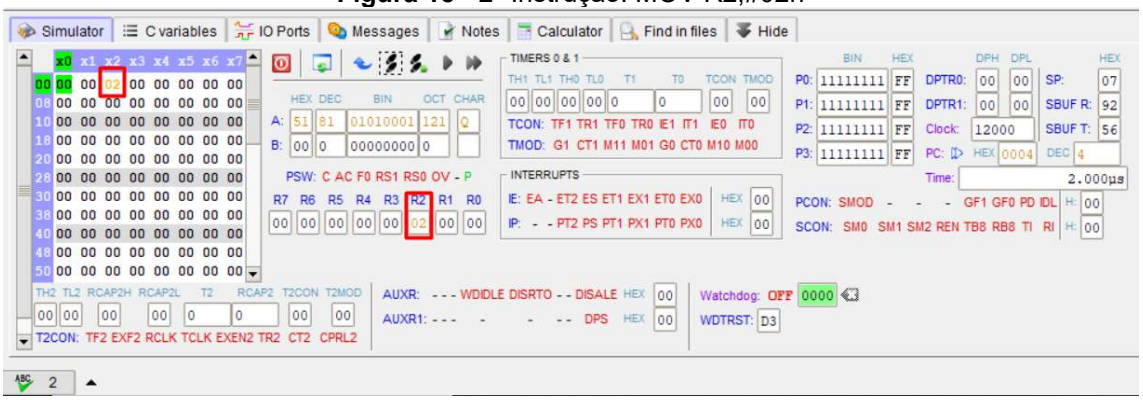
Na **figura 16** temos a sentença ORG 0000H; o ORG inicia o programa. No exemplo, ele inicia o programa no PC = 0. Neste momento, todos os espaços da memória, o compilador e os registradores estão vazios. Também pode-se observar na figura que o PC e o Time estão começando no Zero, todos os endereços estão em 11111111 e o SP (stack pointer) começa no 7h (a cor verde indica a próxima linha a ser executada e a cor amarela vai indicar a que estamos executando).

Figura 17 - 1ª Instrução: MOV A, #051h



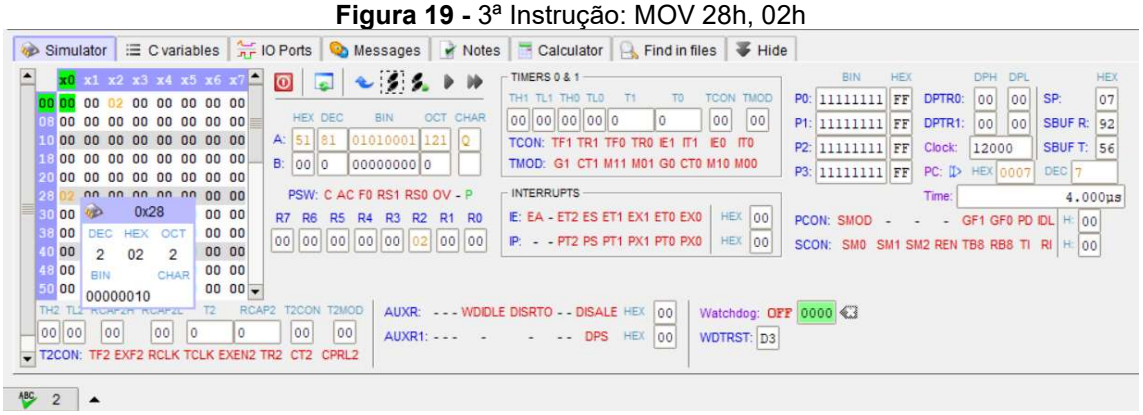
Na **figura 17** temos a sentença MOV A, #051H; que move o dado 51_{16} imediatamente para o Acumulador gastando 2 Bytes e time de 1,0 µs. Alterando o valor de A.

Figura 18 - 2º Instrução: MOV R2,#02h



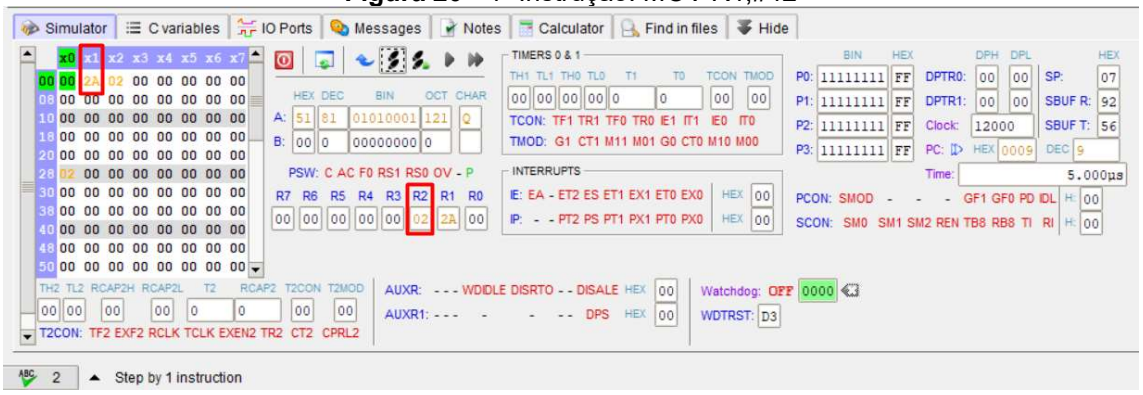
Essa instrução utiliza o endereçamento imediato e registrador para mover o valor de 2_{16} para o R2 gastando 2 Bytes e time de 1,0 µs. Tanto o endereço de memória quanto o registrador mudaram para “02”.

Figura 19 - 3ª Instrução: MOV 28h, 02h



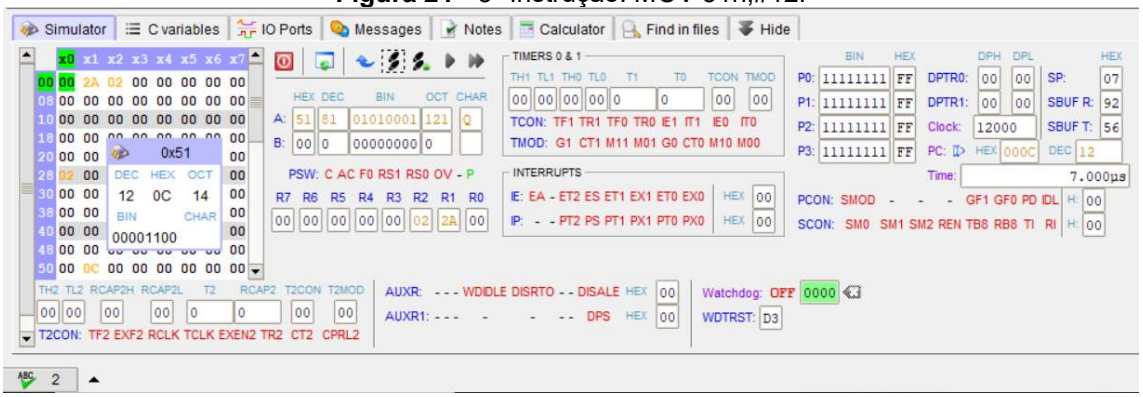
Essa instrução utiliza o endereçamento direto partindo do Endereço de memória 2_{16} (R2), que possuía o valor de 2_{16} para o endereço de memória 28_{16}. Onde os dois endereços agora comportam os valores $2_{16.}$ gastando 3 Bytes e time de 2,0 µs.

Figura 20 - 4ª Instrução: MOV R1,#42



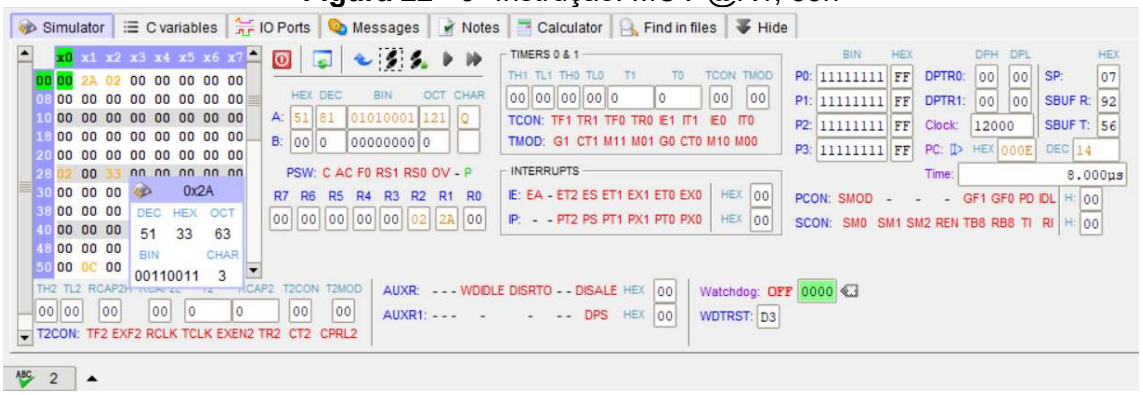
Instrução de endereçamento de Registrador Imediato. Onde é atribuído para o Registrador R2 o valor de 42_{10} quem hexadecimal é representado no $2A_{16.}$ Essa operação gastou 2 Bytes e time de 1,0 µs.

Figura 21 - 5ª Instrução: MOV 51h,#12:



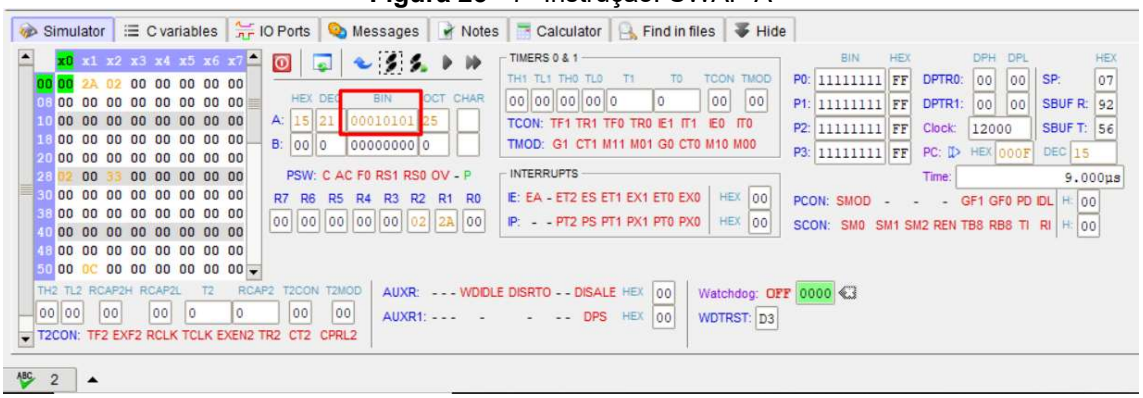
Instrução de endereçamento imediato. Onde foi atribuído para o endereço de memória 51_{16} o valor de 12_{10} ou $0C_{16}$ utilizando 3 Bytes e time de 2,0 µs.

Figura 22 - 6ª Instrução: MOV @R1, 33h



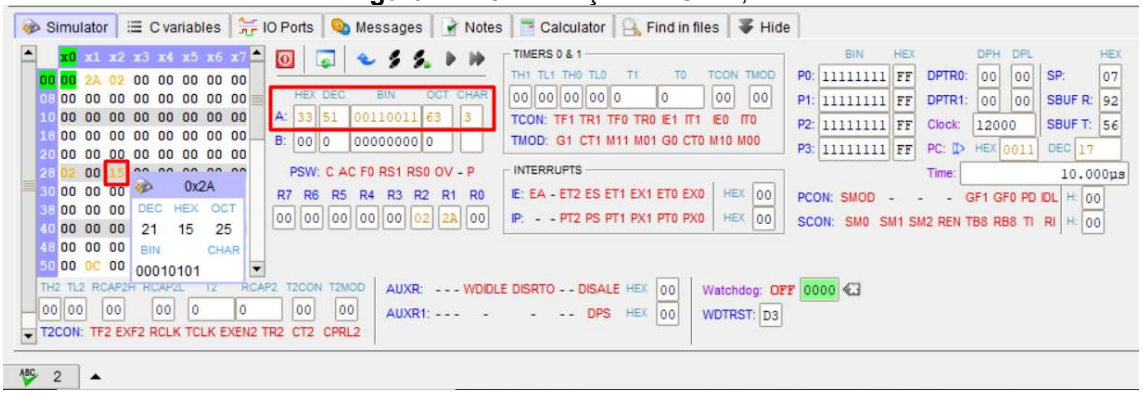
Instrução de endereçamento indireto e imediato. Onde foi atribuído para o endereço de memória $2A_{16}$ o valor de 33_{16}. Nessa operação foi utilizado 2 Bytes e time de 1,0 µs.

Figura 23 - 7ª Instrução: SWAP A



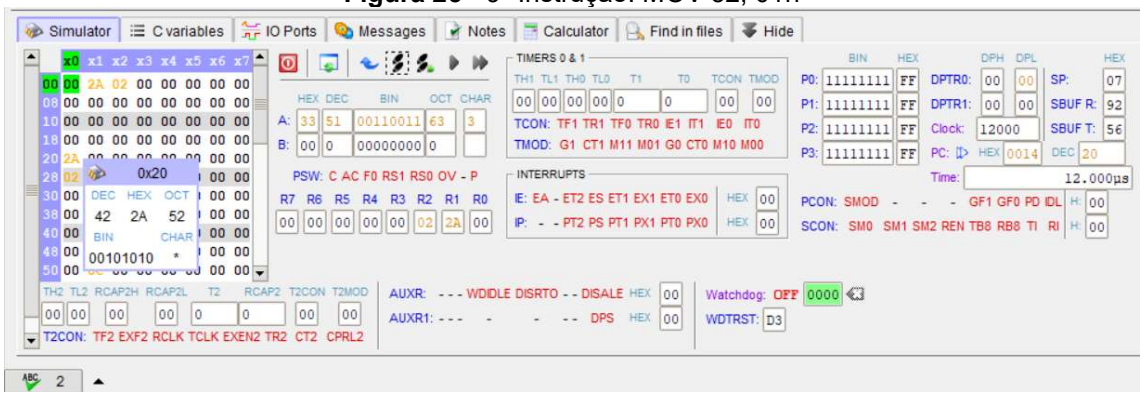
Instrução referente a troca do nibble inferior pelo superior. No caso o valor 0101|0001 no bin foi invertido para 0001|0101. Alterando o valor do acumulador de 51_{16} para 15_{16}. Utilizando nessa operação 1 Byte e time de 1,0 µs.

Figura 24 - 8ª Instrução – XCH A,2Ah



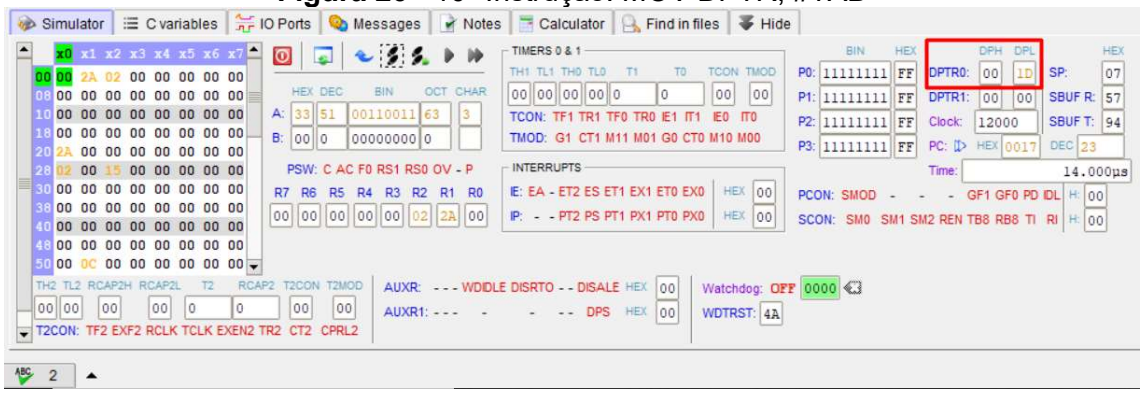
Essa instrução se refere a uma instrução de transferência de dados. A instrução XCH se refere a troca de valores pelo registro. O método utilizado foi o Direto saindo do registro de endereço para $2A_{16}$, para o acumulador A. Desse modo os valores que estavam presentes nesse endereço (33_{16}) foram para o acumulador (15_{16}) e os valores do Acumulador foram para esse endereço. Utilizando 2 Bytes e time de 1,0 µs.

Figura 25 - 9ª Instrução: MOV 32, 01h



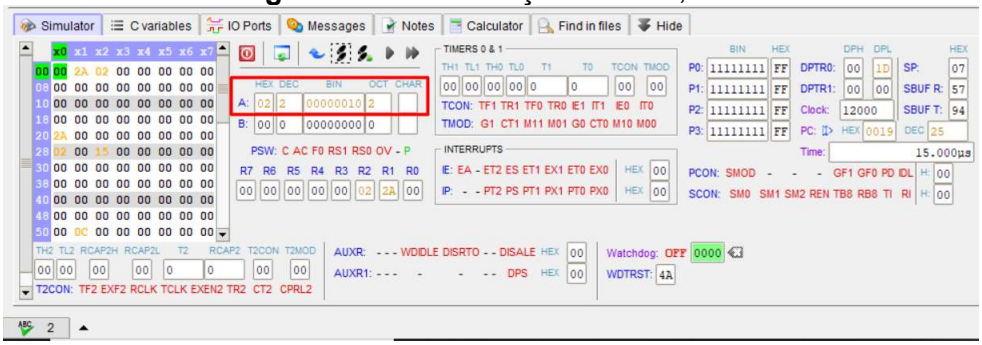
Essa instrução se refere a um endereçamento direto. Movendo os valores de 01_{16} para o endereço de memória 32_{10} ou 20_{16}. Essa operação utilizou 3 bytes e time de 2,0 µs.

Figura 26 - 10ª Instrução: MOV DPTR, #TAB



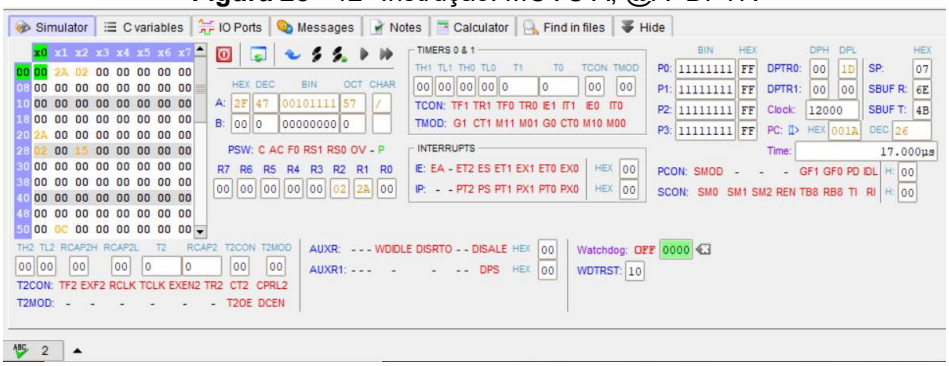
Endereçamento imediato do DPTR com o valor char TAB, onde TAB equivale a $1D_{16}$. Essa operação utilizou 3 bytes e time de 2,0 µs.

Figura 27 - 11ª Instrução: MOV A, 28h



Endereçamento direto do registro de memória de 28_{16} ($02A_{16}$) para o acumulador A. Essa operação utilizou 2 bytes e time de 1,0 µs.

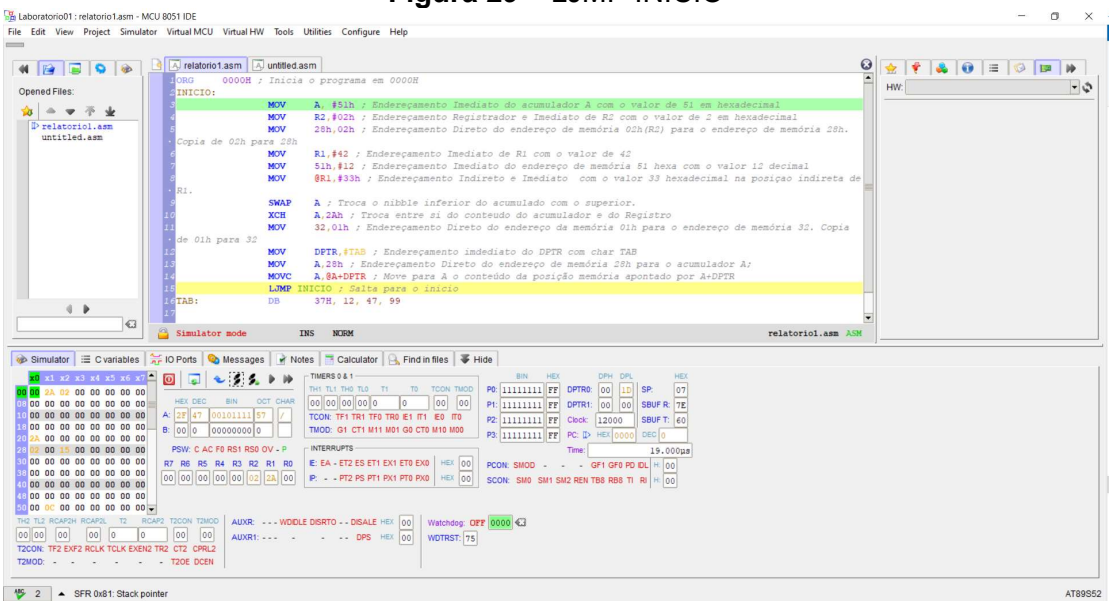
Figura 28 - 12ª Instrução: MOVC A, @A+DPTR



Essa instrução se refere ao modo de endereçamento indexado, onde se soma o valor apontado por A que o da posição 00x1 ($2^a{}_{16}$) mais o valor do DPTR0: DPL ($1D_{16}$) resultando em $2F_{16}$. Onde esse valor é enviado para o acumulado A. Essa operação utilizou 1 byte e time de 2,0 µs.

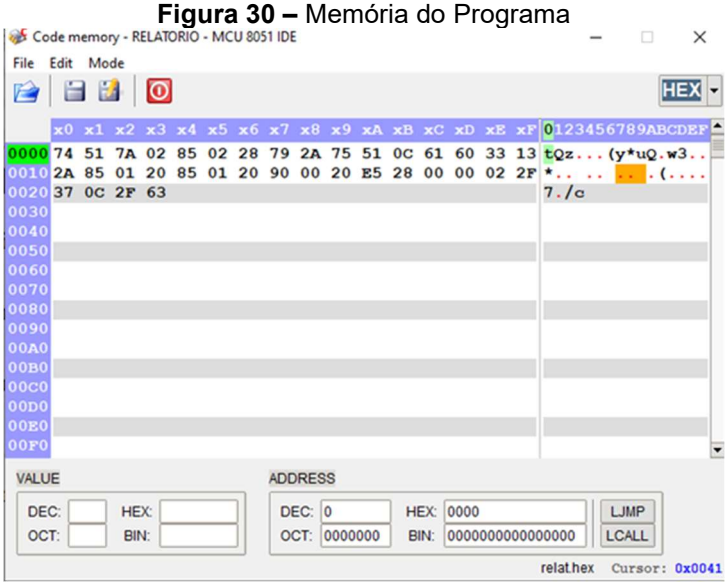
Após essa instrução o programa retorna para primeira instrução apresentando a seguinte situação:

Figura 29 – LJMP INICIO



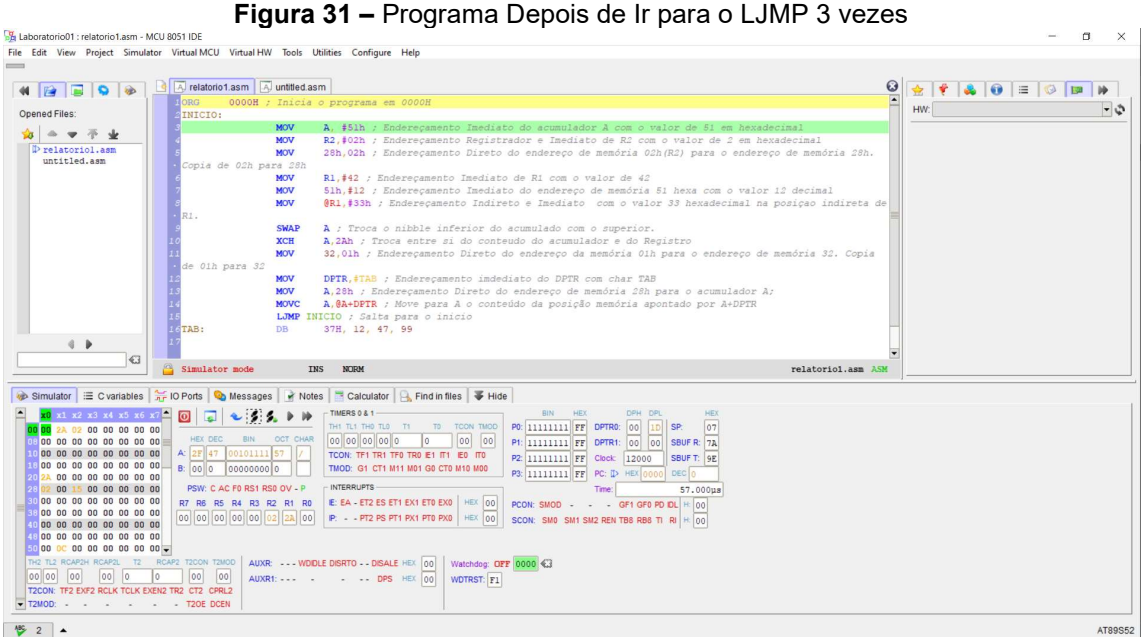
8. O que foi gravado na Memória do Programa:

Figura 30 – Memória do Programa



9. Qual o conteúdo do acumulador quando o PC for 0 (estiver em INICIO) pela terceira vez?

Figura 31 – Programa Depois de Ir para o LJMP 3 vezes



O Conteúdo de PC ao ir para o inicio pela terceira vez será de 0000H. O acumulador A apresentará o valor $2F_{16}$ ao iniciar as instruções.

CONCLUSÃO

As atividades definidas pelo professor nesse primeiro relatório permitiram o desenvolvimento de habilidades para utilização do MCU8051. Onde o contato com o programa auxiliou a visualização das instruções em um ambiente digital. Desse modo foi possível observar sobre uma nova ótica as instruções passadas em sala de aula e verificar o seu comportamento no microcontrolador.